R1CS에서 QAP로의 변환 상세 분석

목차

1. 변환 개요
2. R1CS (Rank-1 Constraint System)
3. QAP (Quadratic Arithmetic Program)
4. 라그랑주 보간법
5. Instance Map 구현
6. Witness Map 구현
7. FFT 기반 최적화
8. 성능 분석 및 최적화

변환 개요

R1CS에서 QAP로의 변환은 Groth16의 핵심 수학적 기반입니다. 이 변환을 통해 제약 조건들을 다항식 형태로 표현하여 효율적인 영지식 증명이 가능해집니다.

변환의 목적

R1CS의 한계:

- 각 제약 조건을 개별적으로 검증해야 함
- 제약 조건 수에 비례하는 검증 시간
- 영지식 증명 구성의 복잡성

QAP의 장점:

- 모든 제약 조건을 단일 다항식 관계로 통합
- 상수 시간 검증 가능
- 페어링 기반 영지식 증명에 적합

변환 과정 개요

```
pub trait R1CSToQAP {
    /// R1CS 인스턴스를 QAP로 변환
    fn instance_map_with_evaluation<F: PrimeField, D: EvaluationDomain<F>>(
    cs: ConstraintSystemRef<F>,
    t: &F, // 평가점 τ
    ) -> Result<(Vec<F>, Vec<F>, Vec<F>, F, usize, usize), SynthesisError>;

    /// R1CS witness를 QAP witness로 변환
    fn witness_map_from_matrices<F: PrimeField, D: EvaluationDomain<F>>(
    matrices: &ConstraintMatrices<F>,
    num_inputs: usize,
    num_constraints: usize,
    full_assignment: &[F],
    ) -> R1CSResult<Vec<F>>;

    /// H 쿼리 스칼라 계산
    fn h_query_scalars<F: PrimeField, D: EvaluationDomain<F>>(
```

```
        max_power: usize,
        t: F,
        zt: F,
        delta_inverse: F,
        ) -> Result<Vec<F>, SynthesisError>;
}
```

R1CS (Rank-1 Constraint System)

R1CS 정의

R1CS는 다음 형태의 제약 조건들로 구성됩니다:

```
제약 조건 i: (Σ a_{i,j} * w_j) * (Σ b_{i,j} * w_j) = (Σ c_{i,j} * w_j)
```

여기서:

- `w_j` : witness 변수들 (w_0 = 1, w_1...w_ℓ = 공개 입력, w_{ℓ+1}...w_m = 비밀 witness)
- `a_{i,j}`, `b_{i,j}`, `c_{i,j}` : 제약 조건의 계수들

R1CS 행렬 표현

```
pub struct ConstraintMatrices<F: Field> {
        /// A 행렬: 각 행은 하나의 제약 조건의 A 부분
        pub a: Vec<Vec<(F, usize)>>, // (계수, 변수 인덱스) 쌍들
        /// B 행렬: 각 행은 하나의 제약 조건의 B 부분
```

```
        pub  b: Vec<Vec<(F, usize)>>,

        /// C 행렬: 각 행은 하나의 제약 조건의 C 부분

        pub  c: Vec<Vec<(F, usize)>>,

        /// 변수 개수

        pub  num_variables: usize,

        /// 제약 조건 개수

        pub  num_constraints: usize,

}
```

R1CS 예시: 곱셈 회로

회로: `x * y = z`

변수 할당:

- `w_0 = 1` (상수)
- `w_1 = z` (공개 입력, 결과값)
- `w_2 = x` (비밀 witness)
- `w_3 = y` (비밀 witness)

제약 조건:

```
w_2 * w_3 = w_1
```

행렬 표현:

```
// A 행렬 (1개 제약 조건)
```

```
a[0] = [(1, 2)] // 1 * w_2


// B 행렬

b[0] = [(1, 3)] // 1 * w_3


// C 행렬

c[0] = [(1, 1)] // 1 * w_1
```

복잡한 예시: 이차 방정식

회로: `x^2 + 3x + 2 = y`

중간 변수 도입:

- `w_0 = 1`
- `w_1 = y` (공개 입력)
- `w_2 = x` (비밀 witness)
- `w_3 = x^2` (중간 변수)

제약 조건들:

```
제약 1: w_2 * w_2 = w_3 (x^2 계산)

제약 2: (w_3 + 3*w_2 + 2*w_0) * 1 = w_1 (최종 계산)
```

행렬 표현:

```
// 제약 1: w_2 * w_2 = w_3

a[0] = [(1, 2)] // w_2
```

```
b[0] = [(1, 2)] // w_2

c[0] = [(1, 3)] // w_3


// 제약 2: (w_3 + 3*w_2 + 2*w_0) * 1 = w_1

a[1] = [(1, 3), (3, 2), (2, 0)] // w_3 + 3*w_2 + 2*w_0

b[1] = [(1, 0)] // 1 (상수)

c[1] = [(1, 1)] // w_1
```

QAP (Quadratic Arithmetic Program)

QAP 정의

QAP는 R1CS를 다항식 형태로 변환한 것입니다:

```
A(x) * B(x) - C(x) = H(x) * Z(x)
```

여기서:

- `A(x)`, `B(x)`, `C(x)` : witness에 대한 다항식들
- `H(x)` : 몫 다항식 (witness가 올바르면 존재)
- `Z(x)` : 소멸 다항식 (회로 구조에 의해 결정)

다항식 구성

각 변수 j에 대해:

```
A_j(x) = Σ(a_{i,j} * L_i(x)) for i = 0, ..., m-1
```

```
B_j(x) = Σ(b_{i,j} * L_i(x))

C_j(x) = Σ(c_{i,j} * L_i(x))
```

여기서 `L_i(x)` 는 i번째 라그랑주 기저 다항식입니다.

전체 다항식

witness `w = (w_0, w_1, ..., w_n)` 에 대해:

```
A(x) = Σ(w_j * A_j(x))

B(x) = Σ(w_j * B_j(x))

C(x) = Σ(w_j * C_j(x))
```

소멸 다항식

```
// Z(x) = Π(x - ω^i) for i = 0, ..., m-1

// 여기서 ω는 m차 단위근, m은 제약 조건 수

fn  vanishing_polynomial<F: PrimeField, D: EvaluationDomain<F>>(

domain: &D,

x: F,

) -> F {

        domain.evaluate_vanishing_polynomial(x)

}
```

소멸 다항식의 성질:

- **도메인에서 0**: `Z(ω^i) = 0` for all i
- **도메인 외부에서 비영**: `Z(τ) ≠ 0` if τ ∉ {ω^0, ..., ω^{m-1}}

라그랑주 보간법

라그랑주 기저 다항식

```
// L_i(x) = Π_{j≠i} (x - ω^j) / (ω^i - ω^j)
// 성질: L_i(ω^j) = δ_{i,j} (크로네커 델타)
```

효율적인 라그랑주 계수 계산

```
// EvaluationDomain 트레이트에서
pub fn evaluate_all_lagrange_coefficients(&self, tau: F) -> Vec<F> {
    let size = self.size();
    let mut coeffs = vec![F::zero(); size];
    // Z(τ) = Π(τ - ω^i) 계산
    let z_tau = self.evaluate_vanishing_polynomial(tau);
    if z_tau.is_zero() {
        // τ가 도메인에 속하는 경우 (일반적이지 않음)
        panic!("Evaluation point is in the domain");
    }
    // 각 i에 대해 L_i(τ) 계산
    for i in 0..size {
```

```
            let omega_i = self.element(i); // ω^i
            // L_i(τ) = Z(τ) / ((τ - ω^i) * Z'(ω^i))
            // 여기서 Z'(ω^i) = Π_{j≠i} (ω^i - ω^j)
            let denominator = (tau - omega_i) * self.evaluate_vanishing_pol
            coeffs[i] = z_tau / denominator;
        }
        coeffs
    }
```

최적화된 구현

```
impl<F: PrimeField> GeneralEvaluationDomain<F> {
        pub fn evaluate_all_lagrange_coefficients(&self, tau: F) -> Vec<F> {
            match self {
                // 2의 거듭제곱 도메인에 대한 최적화
                Self::Radix2(domain) => domain.evaluate_all_lagrange_coe
                // 일반적인 경우
                Self::Mixed(domain) => domain.evaluate_all_lagrange_coef
            }
        }
}

// Radix-2 도메인에 대한 최적화
```

```
fn evaluate_all_lagrange_coefficients_radix2<F: PrimeField>(
&self,
tau: F,
) -> Vec<F> {
        let n = self.size();
        let mut coeffs = vec![F::zero(); n];
        // Z(τ) = τ^n - 1 (2의 거듭제곱 도메인의 특성 활용)
        let z_tau = tau.pow([n as u64]) - F::one();
        // 비트 역순 순서로 계산 (FFT 최적화)
        for i in 0..n {
                let omega_i = self.group_gen.pow([i as u64]);
                let denominator = F::from(n as u64) * (tau - omega_i);
                coeffs[i] = z_tau / denominator;
        }
        coeffs
}
```

Instance Map 구현

기본 구현

```
fn instance_map_with_evaluation<F: PrimeField, D: EvaluationDomain<F>>(
cs: ConstraintSystemRef<F>,
```

```
    t: &F, // 평가점 τ
    ) -> R1CSResult<(Vec<F>, Vec<F>, Vec<F>, F, usize, usize)> {
        // 1. 기본 정보 추출
        let matrices = cs.to_matrices().unwrap();
        let domain_size = cs.num_constraints() + cs.num_instance_variables();
        let domain = D::new(domain_size).ok_or(SynthesisError::PolynomialDegree
        let domain_size = domain.size();

        // 2. 소멸 다항식 Z(τ) 계산
        let zt = domain.evaluate_vanishing_polynomial(*t);

        // 3. 라그랑주 계수들 계산
        let coefficients_time = start_timer!(|| "Evaluate Lagrange coefficients
        let u = domain.evaluate_all_lagrange_coefficients(*t);
        end_timer!(coefficients_time);

        // 4. QAP 다항식 계수들 초기화
        let qap_num_variables = (cs.num_instance_variables() - 1) + cs.num_witn
        let mut a = vec![F::zero(); qap_num_variables + 1];
        let mut b = vec![F::zero(); qap_num_variables + 1];
        let mut c = vec![F::zero(); qap_num_variables + 1];

        // 5. 공개 입력 부분 설정
```

```
{
let  start = 0;
let  end = cs.num_instance_variables();
let  num_constraints = cs.num_constraints();
// 공개 입력들은 제약 조건 이후의 라그랑주 계수들을 사용
a[start..end].copy_from_slice(&u[(start + num_constraints)..(end + num_c
}


// 6. 각 제약 조건에 대해 다항식 계수 계산
for (i, u_i) in  u.iter().enumerate().take(cs.num_constraints()) {
        // A_j(τ) += a_{i,j} * L_i(τ)
        for &(ref  coeff, index) in &matrices.a[i] {
                a[index] += &(*u_i * coeff);
        }
        // B_j(τ) += b_{i,j} * L_i(τ)
        for &(ref  coeff, index) in &matrices.b[i] {
                b[index] += &(*u_i * coeff);
        }
        // C_j(τ) += c_{i,j} * L_i(τ)
        for &(ref  coeff, index) in &matrices.c[i] {
                c[index] += &(*u_i * coeff);
        }
}
```

```
        Ok((a, b, c, zt, qap_num_variables, domain_size))
}
```

공개 입력 처리의 특별한 구조

```
// 공개 입력들은 특별한 방식으로 처리됨
{
let  start = 0;
let  end = cs.num_instance_variables();
let  num_constraints = cs.num_constraints();
// 공개 입력들은 제약 조건들 이후의 도메인 점들에 대응
// 이는 공개 입력이 "가상의 제약 조건"처럼 작동하게 함
a[start..end].copy_from_slice(&u[(start + num_constraints)..(end + num_constrain
}
```

수학적 의미:

- 공개 입력 `w_i` 는 도메인 점 `ω^{m+i}` 에서 값 1을 가지는 다항식에 대응
- 이를 통해 공개 입력들이 QAP 구조에 자연스럽게 통합됨

희소성 최적화

```
// 0이 아닌 원소 개수 계산
```

```
let non_zero_a: usize = cfg_into_iter!(0..qap_num_variables)
.map(|i| usize::from(!a[i].is_zero()))
.sum();


let non_zero_b: usize = cfg_into_iter!(0..qap_num_variables)
.map(|i| usize::from(!b[i].is_zero()))
.sum();
```

희소성 활용:

- 대부분의 제약 조건은 소수의 변수만 사용
- 0이 아닌 계수만 저장하여 메모리 절약
- MSM 계산 시 0인 항목 생략

Witness Map 구현

기본 구현 구조

```
fn witness_map_from_matrices<F: PrimeField, D: EvaluationDomain<F>>(
matrices: &ConstraintMatrices<F>,
num_inputs: usize,
num_constraints: usize,
full_assignment: &[F],
) -> R1CSResult<Vec<F>> {
        let domain = D::new(num_constraints + num_inputs)
```

```
.ok_or(SynthesisError::PolynomialDegreeTooLarge)?;

let domain_size = domain.size();

let zero = F::zero();


// 1. A(x), B(x) 다항식을 평가 형태로 계산

let mut a = vec![zero; domain_size];

let mut b = vec![zero; domain_size];


// 2. 각 제약 조건에서 A, B 값 계산

cfg_iter_mut!(a[..num_constraints])

.zip(cfg_iter_mut!(b[..num_constraints]))

.zip(cfg_iter!(&matrices.a))

.zip(cfg_iter!(&matrices.b))

.for_each(|(((a_val, b_val), a_constraint), b_constraint)| {

*a_val = evaluate_constraint(a_constraint, full_assignment);

*b_val = evaluate_constraint(b_constraint, full_assignment);

});


// 3. 공개 입력 부분 설정

{

let start = num_constraints;

let end = start + num_inputs;

a[start..end].clone_from_slice(&full_assignment[..num_inputs]);
```

```
}

// 4. 계수 도메인에서 평가 도메인으로 변환 (IFFT)
domain.ifft_in_place(&mut a);
domain.ifft_in_place(&mut b);

// 5. 코셋에서 FFT (곱셈을 위해)
let coset_domain = domain.get_coset(F::GENERATOR).unwrap();
coset_domain.fft_in_place(&mut a);
coset_domain.fft_in_place(&mut b);

// 6. A(x) * B(x) 계산 (점별 곱셈)
let mut ab = domain.mul_polynomials_in_evaluation_domain(&a, &b);
drop(a); drop(b); // 메모리 절약

// 7. C(x) 계산
let mut c = vec![zero; domain_size];
cfg_iter_mut!(c[..num_constraints])
.enumerate()
.for_each(|(i, c_val)| {
        *c_val = evaluate_constraint(&matrices.c[i], full_assignment);
});
```

```
        domain.ifft_in_place(&mut c);
        coset_domain.fft_in_place(&mut c);

        // 8. H(x) = (A(x) * B(x) - C(x)) / Z(x) 계산
        let vanishing_polynomial_over_coset = domain
        .evaluate_vanishing_polynomial(F::GENERATOR)
        .inverse()
        .unwrap();
        cfg_iter_mut!(ab).zip(c).for_each(|(ab_i, c_i)| {
                *ab_i -= &c_i; // A*B - C
                *ab_i *= &vanishing_polynomial_over_coset; // / Z
        });

        // 9. 계수 형태로 변환
        coset_domain.ifft_in_place(&mut ab);
        Ok(ab)
}
```

제약 조건 평가 최적화

```
#[inline]
pub fn evaluate_constraint<F: PrimeField>(
terms: &[(F, usize)], // (계수, 변수 인덱스) 쌍들
```

```
    assignment: &[F], // witness 할당

    ) -> F {

            #[cfg(feature = "parallel")]

            if  terms.len() < 100 {

                    serial_evaluate_constraint(terms, assignment)

            } else {

            // 병렬 처리로 성능 향상

            terms.par_iter()

            .map(|(coeff, index)| {

                    let  val = assignment[*index];

                    if  coeff.is_one() {

                            val  // 곱셈 생략

                    } else {

                            val * coeff

                    }

                    })

                    .sum()

            }

            #[cfg(not(feature = "parallel"))]

            serial_evaluate_constraint(terms, assignment)

    }



    fn  serial_evaluate_constraint<F: PrimeField>(
```

```
    terms: &[(F, usize)],

    assignment: &[F]

    ) -> F {

        let mut sum = F::zero();

        // CPU 벡터화를 위해 청크 단위로 처리

        for chunk in terms.chunks(4) {

            let chunk_sum = chunk.iter()

            .map(|(coeff, index)| {

                let val = assignment[*index];

                if coeff.is_one() {

                    val

                } else {

                    val * coeff

                }

            })

            .sum::<F>();

            sum += chunk_sum;

        }

        sum

    }
```

최적화 기법:

1. **특수 케이스**: 계수가 1인 경우 곱셈 생략

2. **청크 처리**: CPU 벡터화를 위한 작은 단위 처리

3. **병렬 처리**: 큰 제약 조건에 대해 Rayon 사용

4. **조기 종료**: 임계값 기반 병렬/순차 처리 선택

FFT 기반 최적화

도메인 변환의 필요성

계수 형태 vs 평가 형태:

- **계수 형태**: 다항식을 계수들로 표현 `p(x) = a_0 + a_1*x + ... + a_n*x^n`
- **평가 형태**: 특정 점들에서의 값으로 표현 `p = [p(ω^0), p(ω^1), ..., p(ω^{n-1})]`

FFT/IFFT 변환 과정

```
// 1. 계수 → 평가 (IFFT)
domain.ifft_in_place(&mut a);
domain.ifft_in_place(&mut b);

// 2. 코셋에서 평가 (FFT)
let coset_domain = domain.get_coset(F::GENERATOR).unwrap();
coset_domain.fft_in_place(&mut a);
coset_domain.fft_in_place(&mut b);

// 3. 점별 곱셈 (O(n) 시간)
let mut ab = domain.mul_polynomials_in_evaluation_domain(&a, &b);
```

```
// 4. 평가 → 계수 (IFFT)
coset_domain.ifft_in_place(&mut ab);
```

코셋 사용 이유

일반 도메인에서의 문제:

```
// 일반 도메인 {ω^0, ω^1, ..., ω^{n-1}}에서
// Z(ω^i) = 0 for all i
// 따라서 (A*B - C) / Z 계산 불가
```

코셋 도메인의 해결책:

```
// 코셋 도메인 {g*ω^0, g*ω^1, ..., g*ω^{n-1}}에서 (g는 생성원)
// Z(g*ω^i) ≠ 0 for all i
// 따라서 나눗셈 가능
let coset_domain = domain.get_coset(F::GENERATOR).unwrap();
let vanishing_over_coset = domain.evaluate_vanishing_polynomial(F::GENERATOR);
```

다항식 곱셈 최적화

```
impl<F: PrimeField> EvaluationDomain<F> {
/// 평가 도메인에서 다항식 곱셈
fn mul_polynomials_in_evaluation_domain(
```

```
    &self,
    a_evals: &[F],
    b_evals: &[F],
    ) -> Vec<F> {
        assert_eq!(a_evals.len(), b_evals.len());
        assert_eq!(a_evals.len(), self.size());
        // 점별 곱셈 (O(n) 시간)
        cfg_iter!(a_evals)
        .zip(cfg_iter!(b_evals))
        .map(|(a, b)| *a * b)
        .collect()
        }
    }
```

시간 복잡도 비교:

- **계수 형태 곱셈**: $O(n^2)$
- **평가 형태 곱셈**: O(n)
- **FFT 변환**: O(n log n)
- **전체**: O(n log n) (FFT가 지배적)

성능 분석 및 최적화

시간 복잡도 분석

| 단계 | 시간 복잡도 | 주요 연산 |

|------|----------|----------|

| 라그랑주 계수 | $O(n^2)$ | 각 계수별 개별 계산 |

| 제약 조건 평가 | O(mn) | m개 제약, n개 변수 |

| FFT 변환 | O(n log n) | 4회 FFT/IFFT |

| 점별 곱셈 | O(n) | 평가 형태 곱셈 |

| **전체** | **$O(n^2 + mn)$** | **라그랑주가 지배적** |

메모리 사용량 분석

```
struct MemoryUsage {
        lagrange_coefficients: usize, // O(n) - 라그랑주 계수들
        polynomial_evaluations: usize, // O(n) - A, B, C 평가값들
        fft_workspace: usize, // O(n) - FFT 작업 공간
        constraint_matrices: usize, // O(mn) - 희소 행렬
        total: usize, // O(n + mn)
}
```

라그랑주 계수 계산 최적화

기본 구현의 문제점:

```
// O(n²) 시간 복잡도
for i in 0..n {
        let mut coeff = F::one();
        for j in 0..n {
```

```
                if i != j {
                        coeff *= (tau - omega_j) / (omega_i - omega_j);
                }
        }
        coeffs[i] = coeff;
}
```

최적화된 구현:

```
// O(n) 시간 복잡도 (2의 거듭제곱 도메인)
fn evaluate_lagrange_coefficients_optimized<F: PrimeField>(
domain: &Radix2EvaluationDomain<F>,
tau: F,
) -> Vec<F> {
        let n = domain.size();
        let mut coeffs = vec![F::zero(); n];
        // Z(τ) = τ^n - 1
        let z_tau = tau.pow([n as u64]) - F::one();
        let n_inv = F::from(n as u64).inverse().unwrap();
        // 각 계수를 O(1) 시간에 계산
        for i in 0..n {
                let omega_i = domain.group_gen.pow([i as u64]);
                coeffs[i] = z_tau * n_inv / (tau - omega_i);
```

```
        }

        coeffs

}
```

병렬 처리 최적화

```
// 제약 조건 평가 병렬화

cfg_iter_mut!(a[..num_constraints])

.zip(cfg_iter_mut!(b[..num_constraints]))

.zip(cfg_iter!(&matrices.a))

.zip(cfg_iter!(&matrices.b))

.for_each(|(((a_val, b_val), a_constraint), b_constraint)| {

*a_val = evaluate_constraint(a_constraint, full_assignment);

*b_val = evaluate_constraint(b_constraint, full_assignment);

});


// FFT 병렬화 (내부적으로 구현됨)

#[cfg(feature = "parallel")]

domain.ifft_in_place(&mut  a); // 병렬 FFT 사용
```

메모리 접근 패턴 최적화

```
// 캐시 친화적인 데이터 레이아웃
```

```
#[repr(C, align(64))] // 캐시 라인 정렬

struct AlignedConstraint {

a_terms: Vec<(F, usize)>,

b_terms: Vec<(F, usize)>,

c_terms: Vec<(F, usize)>,

}



// 순차 접근 패턴 보장

fn  evaluate_constraints_sequential<F: PrimeField>(

constraints: &[AlignedConstraint],

assignment: &[F],

) -> (Vec<F>, Vec<F>, Vec<F>) {

let  mut  a_vals = Vec::with_capacity(constraints.len());

let  mut  b_vals = Vec::with_capacity(constraints.len());

let  mut  c_vals = Vec::with_capacity(constraints.len());

for  constraint  in  constraints {

        a_vals.push(evaluate_constraint(&constraint.a_terms, assignment));

        b_vals.push(evaluate_constraint(&constraint.b_terms, assignment));

        c_vals.push(evaluate_constraint(&constraint.c_terms, assignment));

}

(a_vals, b_vals, c_vals)

}
```

H 쿼리 스칼라 계산

```
fn h_query_scalars<F: PrimeField, D: EvaluationDomain<F>>(
max_power: usize,
t: F,
zt: F,
delta_inverse: F,
) -> Result<Vec<F>, SynthesisError> {
        // H 쿼리: h_i = (τ^i * Z(τ) / δ) for i = 0, ..., max_power-1
        let base_scalar = zt * &delta_inverse;
        let scalars = cfg_into_iter!(0..max_power)
        .map(|i| {
        // τ^i * Z(τ) / δ 계산
                base_scalar * &t.pow([i as u64])
        })
        .collect::<Vec<_>>();
        Ok(scalars)
}
```

최적화 기법:

- **공통 인수 추출**: `Z(τ) / δ` 를 한 번만 계산
- **거듭제곱 최적화**: 이진 지수법 사용
- **병렬 계산**: 각 거듭제곱을 독립적으로 계산

실제 성능 벤치마크

```
// 다양한 회로 크기에 대한 성능 측정
struct R1CSToQAPBenchmark {
        circuit_size: usize,
        lagrange_time: Duration,
        constraint_eval_time: Duration,
        fft_time: Duration,
        total_time: Duration,
}

// 실제 측정 결과 (예시)
let  benchmarks = vec![
        R1CSToQAPBenchmark {
                circuit_size: 1000,
                lagrange_time: Duration::from_millis(5),
                constraint_eval_time: Duration::from_millis(2),
                fft_time: Duration::from_millis(3),
                total_time: Duration::from_millis(10),
        },
        R1CSToQAPBenchmark {
                circuit_size: 10000,
                lagrange_time: Duration::from_millis(50),
```

```
                constraint_eval_time: Duration::from_millis(15),

                fft_time: Duration::from_millis(8),

                total_time: Duration::from_millis(73),

        },

        // ...

];
```

성능 특성:

- **라그랑주 계수**: 회로 크기에 따라 선형 증가 (최적화 후)
- **제약 조건 평가**: 제약 조건 수와 변수 수에 비례
- **FFT**: 회로 크기에 따라 준선형 증가
- **전체**: 대규모 회로에서는 라그랑주 계수 계산이 지배적

R1CS에서 QAP로의 변환은 Groth16의 수학적 핵심이며, 이 변환의 효율성이 전체 시스템의 성능을 크게 좌우합니다. 특히 라그랑주 보간과 FFT 기반 다항식 연산의 최적화가 핵심이며, 이를 통해 대규모 회로에서도 실용적인 성능을 달성할 수 있습니다.